Data Structures

Lecture 21
Problems on Recursion

Prantik Paul [PNP]
Lecturer
Department of Computer Science and Engineering
BRAC University

Problems on Recursion

```python
def fun(x):

    if(x > 0):
        x -= 1
        fun(x)
        print(x , end=" ")
        x -= 1
        fun(x)
```

fun(4)

0 1 2 0 3 0 1

Problems on Recursion

```python
def fun( a, n):
    if(n == 1):
        return a[0]
    else:
        x = fun(a, n - 1)
    if(x > a[n - 1]):
        return x
    else:
        return a[n - 1]
```

fun([12, 10, 30, 50, 100], 5)

100

Problems on Recursion

Delete All Occurrences of an Element in a Linked List

Problems on Recursion

```
def deleteAll(head, elem):
  if head == None:
    return head
  if head.elem == elem:
    head = head.next
    return deleteAll(head, elem)
  head.next = deleteAll(head.next, elem)
  return head
```

Problems on Recursion

Count Consonants in a String

Problems on Recursion

```python
def isConsonant(ch):

    # To handle lower case
    ch = ch.upper()

    return not (ch == 'A' or ch == 'E' or
                ch == 'I' or ch == 'O' or
                ch == 'U') and ord(ch) >= 65 and ord(ch) <= 90
```

Problems on Recursion

```
# To count total number of
# consonants from 0 to n-1
def totalConsonants(string, n):

    if n == 1:
        return isConsonant(string[0])

    return totalConsonants(string, n - 1) + isConsonant(string[n-1])


# Driver code
string = "abc de"
print(totalConsonants(string, len(string)))
```

Problems on Recursion

Print Index of First Consonant in a String

Problems on Recursion

Print Occurrence of a Character in a String

Problems on Recursion

Print Occurrence of All Characters in a String

Problems on Recursion

Print Index of First Uppercase Letter in a String

Problems on Recursion

Print Sum Of Digits in a Number

Problems on Recursion

```
def sumOfDigits(n):
  if n == 0:
    return 0
  else:
    last_digit = n%10
    sum = last_digit + sumOfDigits(n//10)
    return sum
```

Problems on Recursion

Print All Digits in a Number

Problems on Recursion

Reverse a String using Recursion

Problems on Recursion

```
def reverseString(s):
  if len(s) == 0:
    return
  temp = s[0]
  reverseString(s[1:])
  print(temp, end='')

reverseString("hello")
```

Problems on Recursion

```
def reverseString(s, rev = [""]):
  if len(s) == 0:
    return
  temp = s[0]
  reverseString(s[1:], rev)
  rev[0] = rev[0] + temp[0]
  return rev

print(reverseString("hello")[0])
```

Problems on Recursion

Check if a String is Palindrome

Problems on Recursion

```python
def isPalindrome(string):
  if(len(string))<1:
    return True
  else:
    if(string[0])== string[-1]:
      return isPalindrome(string[1:-1])
    else:
      return False
```

Problems on Recursion

Check if a Number is Palindrome

Problems on Recursion

```python
def oneDigit(num):

    return ((num >= 0) and
            (num < 10))
```

Problems on Recursion

```python
def isPalUtil(num, dupNum):

    if oneDigit(num):
        return (num == (dupNum[0]) % 10)

    if not isPalUtil(num //10, dupNum):
        return False

    dupNum[0] = dupNum[0] //10

    return (num % 10 == (dupNum[0]) % 10)
```

Problems on Recursion

```
def isPal(num):
    # make number positive
    if (num < 0):
        num = (-num)
    # seperate copy of num
    dupNum = [num]

    return isPalUtil(num, dupNum)
```